

An AI-Powered Cricket Analytics and Decision Support System for T20 International Cricket

Karthikeyan Ramanathan R

*Assistant Professor
karthikeyan.ramanathan@rajalakshmi.edu.in
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India*

Naren Suriya VS

*UG Scholar
230701203@rajalakshmi.edu.in
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India*

Kanak Anand Thadathil

*UG Scholar
230701139@rajalakshmi.edu.in
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India*

Nitesh Raj M

*UG Scholar
230701214@rajalakshmi.edu.in
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India*

Kashif Nazir

*UG Scholar
230701144@rajalakshmi.edu.in
Department of Computer Science
and Engineering
Rajalakshmi Engineering College
Chennai, India*

Abstract— Cricket produces an enormous amount of structured data about batting, bowling, fielding, player roles, venues, pitch conditions and match situations. In practice this information sits in separate tools, and coaches and analysts have to combine it by hand before every selection or tactical decision. This paper presents CrickIQ, a web-based cricket analytics and decision-support platform that brings these activities into one environment. It offers player analytics, team analysis, Playing XI recommendation, pitch analysis, opponent analysis, player-development analysis, match and tournament management, reporting and conversational assistance. The system is built with a React and TypeScript front end and Supabase/PostgreSQL, and it also supports local CSV datasets and predefined fallback data. Its analytical intelligence is deterministic and does not rely on trained machine-learning models. Player-impact scores combine batting, bowling and fielding statistics, match-winning probability comes from a manually constructed logistic transformation, pitch conditions are evaluated with rule-based calculations, and Playing XI selection uses a greedy, role-constrained heuristic. An optional Gemini integration supports natural-language interaction. CrickIQ should therefore be read as an interpretable decision-support system and not as a trained prediction system. Its main contribution is the integration of many analytical and management functions in one practical application, while keeping the reasoning behind each recommendation transparent. The integrated framework reduces the need for manual cross-referencing of cricket data and provides a consistent workflow for performance analysis and tactical planning. The system also provides a foundation for incorporating more advanced predictive techniques and larger datasets in future implementations. By combining statistical analysis, rule-based reasoning and optimization heuristics, the platform supports multiple decision-making tasks within a unified workflow. The system emphasizes interpretability, allowing users to understand the statistical factors and constraints influencing each recommendation. This approach demonstrates the potential of lightweight, transparent analytics for practical cricket performance analysis and tactical decision-making.

Index Terms—Cricket Analytics, Decision Support, Player Evaluation, Playing XI Recommendation, Rule-Based Intelligence, Generative AI.

I. INTRODUCTION

Cricket is one of the most information-dense team sports. Every match leaves a detailed record of batting and bowling, fielding, scoring events, player roles, match situations, venues and playing conditions. Analysis has traditionally relied on scorecards, individual statistics and the experience of coaches, captains and analysts. That experience remains essential, but the growth of digital cricket data now makes it possible to support those decisions with systematic computation. Runs, balls faced, strike rate, average, wickets, economy, bowling average, catches and phase-wise performance describe what a player can do. Turning those numbers into a decision takes more than displaying them.

Cricket decisions are contextual. A player with excellent overall figures may not be the best option against a particular opponent, on a particular pitch or in a particular phase of an innings. A batter's effectiveness depends on bowling style, handedness, match situation and venue, and a bowler's depends on the pitch, the opposition line-up and the stage of the innings. Team selection adds a further difficulty, because the goal is not simply to pick the eleven best individuals. A competitive Playing XI needs a sensible balance of specialist batters, wicketkeepers, all-rounders, pacers and spinners, and it must account for form, pitch suitability and matchups.

These needs call for decision-support systems that bring several dimensions of cricket information together and present the result in a form people can understand. Computational approaches to sports analytics range from descriptive statistics and mathematical models to machine learning and optimization. Trained models can uncover complex relationships, but they need substantial historical data, careful training and proper validation. They can also be hard to explain to non-technical users, which matters in coaching, where analytics is meant to support human judgment and not replace it.

CrickIQ was built to meet this need for an integrated and interpretable platform. It combines activities that

would normally need separate tools: player evaluation, team analysis, Playing XI recommendation, pitch analysis, opponent analysis, player-development assessment, match analysis, tournament management and reporting. It is a web application using React and TypeScript, with Supabase and PostgreSQL for storage and authentication, and it also supports local CSV datasets and predefined fallback data so that it keeps working when the database is unavailable.

An inspection of the source code shows that the core intelligence of CrickIQ is deterministic. It is made up of statistical calculations, mathematical formulas, heuristic optimization and explicit cricket rules. The player-impact mechanism combines batting, bowling and fielding contributions into a composite score. The match predictor builds a manually defined score from team rating, squad strength, home advantage and pitch contribution and then applies a logistic transformation. Pitch analysis uses explicit characteristics such as grass, moisture, hardness, cracks and bounce, and Playing XI recommendation applies a greedy selection process subject to role requirements.

This distinction needs to be stated plainly. The implementation does not contain trained Random Forest, Logistic Regression, XGBoost, LightGBM, neural-network or deep-learning models. Using the logistic function does not make the predictor a trained logistic regression, because no coefficients are learned from historical outcomes. In the same way, a rule-based recommendation engine is not a machine-learning recommender. Describing the system accurately also makes its real strengths clearer.

The platform does include an optional generative-AI component based on Google Gemini 1.5 Flash. It accepts user prompts and returns natural-language responses, but it is separate from the deterministic engine and does not train models or calculate the core scores and Playing XI recommendations. A local assistant based on keywords, patterns and intent recognition is also provided.

The contributions of this work are as follows:

- 1) An integrated web platform that covers player, team, pitch, opponent, selection, development, match, tournament and reporting functions.
- 2) A transparent, deterministic analytical engine whose outputs can be traced to known inputs and formulas.
- 3) A resilient data design that falls back from Supabase to local CSV files and predefined data.
- 4) A clear statement of the system's limits, together with a foundation for later comparison against trained predictive models.

The work does not introduce a new trained machine-learning algorithm. Its value lies in showing how statistical analysis, mathematical modeling, rule-based reasoning, heuristic optimization and optional generative AI can be combined into a single interpretable cricket decision-support platform.

II. LITERATURE SURVEY

Cricket analytics has moved from reading scorecards toward statistical and computational methods, covering player evaluation, match forecasting, remaining-run prediction, performance prediction and squad selection. Saikia, Bhattacharjee and Mukherjee formulated cricket performance management mathematically and showed that player contributions and team selection can be expressed as quantitative variables [1]. This is close in spirit to the impact score and selection logic in CrickIQ, where raw statistics are reduced to decision-support scores.

Asif and McHale developed an in-play win-probability model for One-Day Internationals using dynamic logistic regression [2]. CrickIQ also uses a logistic function for match probability, but the two are different in an important way. A trained logistic regression learns its coefficients from historical data, whereas CrickIQ builds its input score by hand from team rating, squad strength, home advantage and pitch contribution. For that reason we call it a logistic probability transformation and do not claim it is logistic regression.

Many studies apply machine-learning algorithms to outcome prediction in T20, ODI and IPL cricket [3]–[6], [9], and recurrent models such as LSTM have been used for ball-by-ball win probability [7], [8]. Survey-style overviews are given in [10]. These methods can learn non-linear relationships that hand-written rules miss, but they need large datasets, careful validation and suitable metrics, and CrickIQ does not yet have such a training pipeline.

Squad selection is naturally a constrained optimization problem, since a team must meet role requirements while maximizing expected performance. Researchers have combined performance prediction with optimization and heuristic techniques [11]–[13]. Swarna et al. rank players with an impact-point system and recommend a national squad by role [11]. Banerjee et al. propose heuristic rankings with greedy team-selection algorithms for the IPL [12], and Prakash et al. build random-forest performance indices for IPL team selection [13]. CrickIQ faces the same practical problem and uses a simple greedy algorithm, closest in spirit to [11] and [12]. It is light and easy to explain, but it does not guarantee a globally optimal team.

Research gap. Most published work concentrates on one problem at a time, such as match prediction, score forecasting, performance prediction or squad selection. CrickIQ integrates several decision-support functions in a single web application. At the same time, there is a clear distance between CrickIQ and research-grade machine-learning systems: it has no trained model and no empirical evaluation on unseen matches, and it has no verified live-data feed, although its database schema includes match and ball-level tables. Its strength is interpretability and integration, not learned predictive accuracy.

III. EXISTING SYSTEM AND PROBLEM IDENTIFICATION

Today, cricket analysis falls into roughly three types: manual analysis using scorecards, spreadsheets and anecdotal evidence; aggregation and presentation of historical data from public sources or commercial products; and research programs that use machine-learning or conventional statistical methods. Each has strengths and weaknesses, and together they point toward the need for a more complete solution for coaching and selection decisions.

Four concrete problems stand out. First, evaluation criteria and selection parameters are applied in different ways, so it is often hard to say why one player was chosen over another. Second, context such as the pitch, the opponent and the stage of the innings is rarely built into the selection process and is usually applied afterwards by hand. Third, few tools consider the depth of players across different positions when they build a team. Fourth, useful features are scattered across tools, so users keep moving information from one platform to another. These problems create room for a unified platform that combines player analysis, contextual analysis, selection, management and reporting, and that explains the reasoning behind its recommendations.

IV. PROPOSED METHODOLOGY

CrickIQ follows an end-to-end layered workflow. Data enters through an ingestion layer that handles players,

teams, venues, matches, ball-level events, phase-wise statistics and auction history. It is loaded from Supabase/PostgreSQL when the database is available, and from local CSV files otherwise. The data is then parsed, converted to numeric types and used to calculate derived statistics before it reaches the analytical engine. The engine is a set of deterministic modules, namely mathematical formulas, heuristics and explicit rules, so every output can be traced back to known inputs. This is what makes the system interpretable.

A. Data Ingestion

The current prototype uses local datasets for players, teams, venues, matches, ball-by-ball events, phase statistics and auction history, summarized in Table I. They are enough to demonstrate how the application works, but they cover only a small part of all cricket records. The system also does not connect to a verified live cricket-data API, so the presence of a Supabase database does not mean that data is continuously synchronized from an external provider. Public datasets, such as the Asia Cup 2025 T20 dataset [14], are an example of data that could be used in future.

TABLE I
DATASETS USED IN CRICKIQ

Dataset	Contents	Records
Players	Role, styles, career statistics	[Insert]
Teams	Squads, ratings	[Insert]
Venues	Pitch type, scoring trends	[Insert]
Matches	Teams, venue, toss, result	[Insert]
Ball-by-ball	Delivery-level events	[Insert]
Phase statistics	Powerplay, middle, death	[Insert]
Auction history	Prices and player value	[Insert]

B. Preprocessing and Derived Statistics

Fields are parsed and converted to numeric types, and invalid or missing values are handled before analysis. Derived statistics are then computed from raw counts: strike rate is runs scored divided by balls faced multiplied by 100, economy rate is runs conceded per over, and bowling average is runs conceded per wicket taken. These values form the structured feature set used by the analytical modules.

C. Player Impact Score

The player-impact mechanism combines batting, bowling and fielding contributions into one composite score. Conceptually, for player i ,

$$I_i = w_b B_i + w_w W_i + w_f F_i \quad (1)$$

where B_i , W_i and F_i are the batting, bowling and fielding contributions and w_b , w_w and w_f are fixed weights defined in the application. The weights are set by design and are not learned from data.

D. Pitch and Condition Analysis

Pitch analysis uses explicit characteristics such as grass, moisture, hardness, cracks and bounce, and converts them into tactical information with rule-based calculations. For example, more grass cover and moisture favour seam bowling, a hard surface with good bounce favours pace and stroke play, and a dry, worn surface favours spin. The rules are visible and can be inspected, which lets a coach see why a particular tactical suggestion was made.

E. Playing XI Selection

Playing XI recommendation is a greedy, role-constrained heuristic. The available players are ranked by their scores, and the algorithm repeatedly adds the best remaining player who still fits the role requirements, such as the minimum number of batters, a wicketkeeper, all-rounders, pacers and spinners, until eleven players are chosen. At each step it picks

$$p^* = \operatorname{argmax}_p I_p, p \text{ in } A \quad (2)$$

where A is the set of remaining players whose role is still needed. This approach is quick and easy to explain, but it does not guarantee the globally optimal team.

F. Match-Winning Probability

The match predictor first builds a manual score from four factors:

$$z = a_1 R + a_2 S + a_3 H + a_4 C \quad (3)$$

where R is team rating, S is squad strength, H is home advantage and C is the pitch contribution, and the

coefficients a_1 to a_4 are set manually. A logistic transformation then turns the score into a probability:

$$P(W) = 1 / (1 + e^{-Z}) \quad (4)$$

Because no coefficient is learned from historical outcomes, this is a logistic probability transformation and not a trained logistic regression.

G. Opponent and Development Analysis

Opponent analysis adjusts player evaluation for the opposing side and highlights matchups. Player-development analysis compares phase-wise statistics against fixed thresholds to identify strengths and weaknesses in specific phases of the game. Auction value estimation and team-balance checks use the same deterministic style.

H. Conversational Assistance

A local assistant uses keywords, patterns and intent recognition to answer questions about players, teams and pitches. An optional integration with Google Gemini 1.5 Flash provides generative responses. This component is independent of the analytical engine and does not compute scores or recommendations.

V. SYSTEM DESIGN AND IMPLEMENTATION

CrickIQ uses a modular web architecture. The front end handles the interface and visualization, the application layer holds the analytical modules, and the data layer provides persistent storage with fallbacks. The system is divided into modules for dashboards, players, teams, selection, pitch analysis, opponent analysis, development, matches, live scoring, tournaments, reports and assistance.

A. System Architecture

The architecture, shown in Fig. 1, has a React and TypeScript front end, a deterministic analytics engine, and a data layer made up of Supabase/PostgreSQL, local CSV datasets and predefined fallback data. The optional Gemini assistant connects to the front end and is kept separate from the engine. Supabase provides authentication, and role-based access limits what each user can see and do.

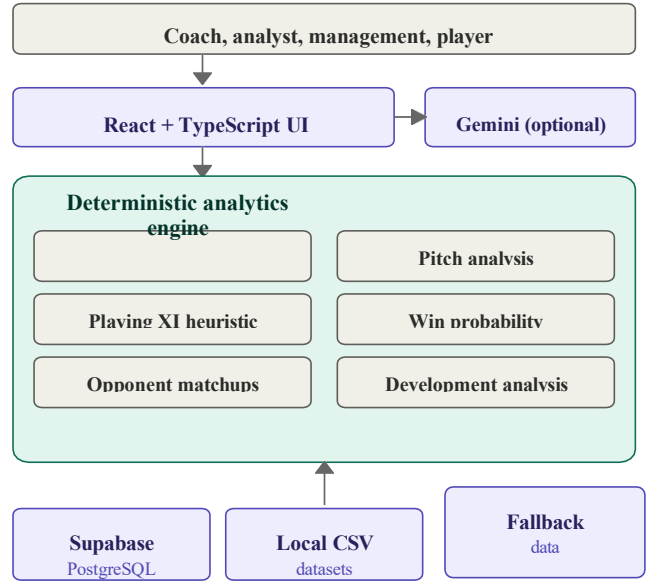


Fig. 1. Architecture of the proposed CrickIQ platform.

B. Technology Stack

The platform uses a web-based stack, listed in Table II. React and TypeScript build the interface, TanStack Start provides the application framework, Tailwind CSS handles styling, and Recharts draws the charts. Supabase supplies PostgreSQL and authentication. Zod validates input, jsPDF and XLSX generate reports, and Vite is the build tool.

TABLE II
TECHNOLOGY STACK

System Layer	Technology / Component
Frontend	React and TypeScript
Framework	TanStack Start
Styling	Tailwind CSS
Visualization	Recharts
Database	PostgreSQL via Supabase
Authentication	Supabase Authentication, role-based access
Local data	CSV datasets and fallback data
Validation	Zod
Reports	jsPDF and XLSX
Build tool	Vite
Generative AI	Google Gemini 1.5 Flash (optional)

C. Analytical Modules

The dashboard summarizes players, teams and matches. The analytics module shows player-impact scores and comparisons with charts. The selection module generates a Playing XI under the role constraints, the pitch module converts the entered conditions into tactical advice, and the opponent module highlights matchups. The development module identifies phase-specific strengths and weaknesses, and the prediction module reports the match-winning probability.

D. Match, Tournament and Reporting Functions

Beyond analysis, CrickIQ supports match management, live scoring and tournament management. Users can record scoring events, view standings and fixtures, and export reports in PDF or spreadsheet form for coaching meetings and reviews.

E. Authentication and Data Resilience

Supabase Authentication handles login and role-based access for coaches, analysts, management and players. For resilience, the data layer tries Supabase/PostgreSQL first, then local CSV files, and finally predefined fallback data, so the application stays usable when the database cannot be reached.

F. System Workflow

A typical session begins with login. The platform loads data from the best available source, derives the statistics, and lets the user explore players and teams. The user sets the pitch conditions and the opponent, and the engine computes impact scores, a Playing XI and a winning probability. The user reviews these outputs, can export a report, and makes the final decision.

VI. RESULTS AND DISCUSSIONS

The implemented platform covers the main activities needed for cricket analysis and decision support in one application. Player statistics are processed into impact scores, pitch conditions are turned into tactical guidance, and a Playing XI can be generated under role constraints. Opponent matchups, development analysis and match-winning probability add further context, while the match, tournament and reporting modules handle management tasks. Table III lists the modules and the focus of their functional validation.

TABLE III
FUNCTIONAL VALIDATION OF THE IMPLEMENTED PLATFORM

Module	Validation Focus
Authentication	Login and role-based access
Data loading	Database, CSV and fallback sources
Player analytics	Impact score calculation
Team analysis	Squad composition and balance
Playing XI	Role-constrained greedy selection
Pitch analysis	Rule-based tactical output
Opponent analysis	Matchup and threat identification
Match prediction	Logistic probability output
Player development	Phase-wise strengths and weaknesses
Matches and scoring	Match records and live scoring events
Tournaments	Fixtures and standings
Reports	PDF and spreadsheet generation
AI assistant	Local and optional Gemini responses

A. Functional Validation

Each module was evaluated against its intended function, mainly to check that it does what it is designed to do and works smoothly with the rest of the workflow. The validation is functional and not statistical. It confirms that the modules work, but it does not measure predictive accuracy on unseen matches.

B. Discussion

The main strength of CrickIQ is its integration of analysis and management in one place, together with the transparency of its reasoning. Because every score comes from a formula or a rule, a coach can trace a recommendation back to its inputs and can question it. This is valuable in coaching, where people are more willing to act on advice they can understand.

The same design involves a trade-off. A deterministic engine cannot learn from data, so its accuracy depends on how well the formulas and weights reflect real cricket. A trained model could capture non-linear relationships that hand-written rules miss, but it would be harder to explain and would need much more data and validation. CrickIQ chooses interpretability first, and it leaves a comparison between the two approaches as future work.

C. Limitations

The system has the following limitations. It contains no trained machine-learning model and has not been empirically evaluated on unseen matches. The datasets are small compared with all available cricket records, and there is no verified live-data feed. Impact weights and probability coefficients are set manually. The greedy Playing XI heuristic does not guarantee an optimal team, and the optional Gemini assistant is not part of the analytical core. Finally, the quality of every output depends on the completeness and accuracy of the input data.

VII. CONCLUSION

This paper presented CrickIQ, a web-based cricket analytics and decision-support platform for T20 international cricket. It brings player analytics, team analysis, Playing XI recommendation, pitch analysis, opponent analysis, player-development analysis, match and tournament management, reporting and conversational assistance into one application built on React, TypeScript and Supabase, with local data and fallback support.

The analytical engine is deterministic: impact scores, a hand-built logistic probability, rule-based pitch analysis and a greedy role-constrained selection heuristic. Describing it this way is accurate, and it highlights the platform's main strength, which is that its recommendations are interpretable. CrickIQ does not claim learned predictive accuracy. It offers a practical, transparent base on which more advanced predictive models can later be built and compared.

VIII. FUTURE SCOPE

Several extensions are possible. First, trained models such as Random Forest, Logistic Regression and XGBoost could be built on larger historical datasets and compared with the deterministic engine through experiments on unseen matches. Second, Playing XI selection could use formal optimization methods, such as integer programming, in place of the greedy heuristic. Third, integration with a verified live-data API would give automatic updates of fixtures, scores and ball-by-ball data. Fourth, the impact weights and probability coefficients could be calibrated or learned from data. Finally, larger user studies with coaches and analysts would show how much the platform improves real decisions.

REFERENCES

- [1] H. Saikia, D. Bhattacharjee, and D. Mukherjee, *Cricket Performance Management: Mathematical Formulation and Analytics*. Springer, 2019. [Verify details]
- [2] R. Asif and I. G. McHale, “In-play forecasting of win probability in One-Day International cricket: A dynamic logistic regression model,” *Int. J. Forecasting*, vol. 32, no. 1, pp. 34–43, 2016. [Verify details]
- [3] [Insert full citation: machine-learning outcome prediction in T20/ODI/IPL cricket.]
- [4] [Insert full citation: machine-learning outcome prediction in T20/ODI/IPL cricket.]
- [5] [Insert full citation: machine-learning outcome prediction in T20/ODI/IPL cricket.]
- [6] [Insert full citation: machine-learning outcome prediction in T20/ODI/IPL cricket.]
- [7] [Insert full citation: recurrent (LSTM) ball-by-ball win-probability model.]
- [8] [Insert full citation: recurrent (LSTM) ball-by-ball win-probability model.]
- [9] [Insert full citation: machine-learning outcome prediction in T20/ODI/IPL cricket.]
- [10] [Insert full citation: survey of cricket analytics and prediction.]
- [11] Swarna et al., [insert title, venue, year]: impact-point player ranking and national squad recommendation by role.
- [12] Banerjee et al., [insert title, venue, year]: heuristic rankings and greedy team selection for the IPL.
- [13] Prakash et al., [insert title, venue, year]: random-forest performance indices for IPL team selection.
- [14] [Insert dataset citation: Asia Cup 2025 T20 dataset, source and access date.]